Safe Tool-Calling Travel Booking Agent with Allow-Lists

Shaila Bhanu Shaik (2520030299)
Abhiram (2520030170)
Generative AI and AI Platform Engineering

Abstract— Tool-calling AI agents can automate travel-related tasks such as searching flights and hotels, checking booking status, and managing reservations. However, an agent that executes every tool request directly may be vulnerable to prompt injection, invalid arguments, unintended actions, and data leakage. This paper presents a safe travel booking agent that places an allow-list and validation layer between the language model and tool execution. Only explicitly approved tools can run; their arguments are checked against structured constraints, and sensitive actions such as booking or cancellation require user confirmation. The system uses mock travel data to avoid real-world transactions and records permitted and rejected requests for auditing. A test plan covering normal travel requests and adversarial inputs is proposed to compare an unrestricted agent with the guarded agent. The approach aims to preserve useful travel assistance while reducing unsafe tool execution.

Index Terms— Generative AI, tool calling, allow-list, agent safety, Pydantic, prompt injection, travel booking.

I. INTRODUCTION

Large language models (LLMs) can interpret user requests and select tools to complete tasks. In a travel application, tools may search flights, search hotels, retrieve booking information, book a flight, or cancel a reservation. Tool calling makes the agent useful, but it also creates a boundary between model-generated decisions and actions that affect application data.

A conventional tool-calling agent may execute a tool request without an independent policy check. A malicious or confusing instruction can therefore cause the model to request an unsafe action or provide incorrect arguments. The proposed Safe Tool-Calling Travel Booking Agent introduces a deterministic guard layer that checks every proposed call before execution. The central principle is that the model may suggest an action, but application code decides whether that action is permitted.

II. PROBLEM STATEMENT

The system must support common travel-assistance tasks while preventing unauthorized or malformed tool calls. The main concerns are tool misuse, argument manipulation (for example, an excessive passenger count), direct and indirect prompt injection, unauthorized booking changes, and exposure of another user's booking details. The design therefore requires explicit tool permissions, schema-based argument validation, confirmation for high-impact actions, safe rejection, and audit logging.

III. RELATED WORK AND BACKGROUND

Tool-using LLM agents combine language understanding with external functions or APIs. Function schemas describe the inputs a tool accepts, while structured validation can reject values that do not satisfy application constraints. Allow-lists apply a default-deny policy: only named tools are eligible for execution. Human-in-the-loop confirmation is commonly used for actions with meaningful consequences. Prompt-injection

testing and regression suites help evaluate whether an agent follows its intended boundaries. These concepts motivate a layered design in which model output is treated as a proposal rather than trusted execution.

IV. PROPOSED SYSTEM

The proposed system contains five main components: (1) a chat interface that accepts travel requests; (2) an LLM client that interprets the request and proposes a tool call; (3) an allow-list and validation guard; (4) a tool executor backed by mock travel data; and (5) a logging and evaluation component. The guard checks tool names, validates arguments using Pydantic-style contracts, enforces limits, and asks for confirmation before booking or cancellation. Calls that fail a check are not executed and receive a safe fallback response.

User request	↓
LLM / Agent	Proposes a tool + arguments
Safety Guard	Allow-list • schema validation • limits • confirmation
Decision	Approved → execute | Rejected → block + log
Mock Travel Tools	Search flights / hotels • booking status • booking actions
Response + Audit Log	Return result to user and record decision

A. Allowed Tools and Policy

The initial allow-list includes search_flights, search_hotels, and get_booking_status. book_flight and cancel_booking may be included as controlled tools, but they require explicit user confirmation and validated parameters. Unknown tools are denied by default. Example constraints include valid airport codes, future travel dates, a bounded passenger count, and a maximum permitted price.

B. Execution Flow

The user submits a request; the LLM selects a tool and constructs its arguments. The guard first checks whether the tool is allowed, then validates the argument schema and business limits. If the action is sensitive, the system requests confirmation. Only an approved call reaches the executor. The result is returned to the model or user, and the decision is logged. Rejected calls return a safe message without modifying booking data.

V. IMPLEMENTATION PLAN

The prototype can be implemented in Python. Pydantic models define tool input contracts; a configuration file stores the allow-list and limits; SQLite or in-memory mock data provides travel records; and a simple Streamlit or web chat interface supports interaction. The LLM client can be hosted or local, while a provider-independent interface keeps the rest of the application reusable. Logs should record the requested tool, validation outcome, confirmation state, and execution result without storing sensitive secrets.

VI. SECURITY TESTING AND EVALUATION

Two variants should be evaluated with the same golden test set: (A) an agent without the guard and (B) the proposed guarded agent. Normal scenarios include flight search, hotel search, and booking-status requests. Adversarial scenarios include instructions to book an excessive number of tickets, invoke an unlisted tool, bypass confirmation, reveal another user’s booking, or follow malicious instructions embedded in tool output. Results should be reported from actual runs rather than assumed.

Metric	Meaning
Task success rate	Percentage of normal requests completed correctly
Allow-list block rate	Percentage of unauthorized tool calls rejected
Attack success rate	Percentage of adversarial cases that cause an unsafe action
Validation rejection count	Invalid arguments stopped before execution
Confirmation compliance	Sensitive actions executed only after approval

VII. EXPECTED OUTCOME

The guarded design is expected to prevent execution of tools outside the allow-list, reject malformed or out-of-range inputs, and add a confirmation step for high-impact booking actions. It should also provide traceable evidence through logs and repeatable tests. The final report should include measured results, a system architecture diagram, the policy configuration, representative blocked attacks, and a comparison between the two agent variants.

VIII. CONCLUSION

This paper describes a travel booking agent that separates the LLM's decision-making from tool execution. An allow-list, structured argument validation, confirmation gates, and audit logging form a practical safety boundary around travel tools. By using mock data and a repeatable test suite, the project can demonstrate both useful travel assistance and safer behavior under adversarial inputs. Actual evaluation numbers will be added after implementation and testing.

REFERENCES

[1] OpenAI, "Function calling and tools," OpenAI API Documentation.

[2] Pydantic, "Data validation using Python type annotations," Pydantic Documentation.

[3] OWASP, "Top 10 for Large Language Model Applications," OWASP GenAI Security Project.

[4] LangGraph, "Human-in-the-loop," LangGraph Documentation.

Note: This is a project research-paper draft. Implementation-specific screenshots, measured metrics, and final experimental results should be inserted after the prototype is completed.